



Practica #1

Allan Daniel Mendez Pum

202330157



Introducción

El manual técnico detalla la arquitectura y el funcionamiento de la primera fase para el lenguaje PromptZal: el analizador léxico. El objetivo de esta fase es leer un programa escrito en PromptZal y leerlo letra por letra (tokens). El desarrollo de este analizador fue realizado utilizando el lenguaje de programación Java.

Lista de Tokens del Lenguaje PromptZal

La siguiente lista de tokens sirve para reconocer las siguientes categorías e ignorando los comentarios de líneas (//) y de bloque (/* */)

- **Directivas:** Elementos de configuración inicial como (@modelo, @rol y @formato.)
- **Palabras reservadas:** Palabras de estructura como (AGENTE, contexto, variable, EJECUTAR y EXPORTAR.)
- **Comandos de IA:** Palabras clave de acción como (PREGUNTAR, GENERAR, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR y EXTRAER.)
- **Conectores:** Símbolos y palabras de unión como (SOBRE, DESDE, EN, COMO y ->.)
- **Identificadores:** Cadenas de texto que inician con (una letra o guion bajo, seguidas por una combinación de letras, dígitos o guiones bajos.)
- **Literales:** Cadenas de texto encerradas entre comillas dobles, así como números numéricos enteros y decimales.



- **Operadores y Delimitadores:** Símbolos para asignación (=), concatenación (+), agrupación con llaves ({ }), paréntesis (()), comas (,) y las comillas de las cadenas.
- **Funciones:** Únicamente la instrucción CARGAR.

Logica del Recorrido

El proceso fue garantizando por el control total sobre la lectura del archivo.

- El analizador realiza el recorrido del archivo de entrada carácter por carácter de forma manual.
- Para cumplir con los lineamientos, no se utilizaron generadores de analizadores léxicos externos ni la librería de expresiones regulares nativa del lenguaje.
- El control del recorrido se maneja mediante un ciclo while y el uso del método `charAt(posicion)`. Esto permite extraer el carácter actual y utilizar un bloque de evaluación condicional que delega el trabajo a submétodos especializados (como `extraerPalabra()` o `extraerNumero()`), protegiendo el encapsulamiento de la clase.



Manejo de Errores Léxicos

El sistema garantiza que el flujo de ejecución no se interrumpa ante equivocaciones en el código fuente.

- Si el programa lee un carácter que no coincide con los patrones definidos, o si se encuentra una cadena de texto sin cerrar, el evento es detectado como un error léxico.
- El análisis léxico no se detiene al hallar un error; el programa simplemente lo reporta y continúa evaluando el texto hasta llegar al final del archivo.
- Se genera una tabla HTML detallando el carácter defectuoso, la fila y la columna del incidente.